Rapport de Mini-Projet: Interpréteur Étendu

fabriqué par: Ahmed Riadh Hedhili
Mohamed Ali Khemiri
Mohamed Aziz Trabelsi

Supervisé par: Mr. Foued Oueslati



1. Introduction

Ce projet consiste à étendre un interpréteur d'expressions arithmétiques et logiques développé avec **Flex** et **Bison**. L'objectif était d'ajouter des fonctionnalités avancées, telles que **les opérateurs de comparaison** et **de logique**, tout en conservant la structure de base fournie dans le template.

2. Fonctionnalités Implémentées

Fonctionnalité	Description
Commentaires	Support des commentaires sur une ligne commencant par **//**
Opérateurs de comparaison	Ajout des opérateurs de base **==** , **!=** , **<** , **>** , **<=** , **>=** .
Opérateurs logiques	Ajout des opérations booléennes **&& (ET)**, **|| (OU)**, **! (NON)**
Boucles	Implémentation des structures itératives **tant que (while)** et **répéter**.
Gestion d'erreurs	Affichage précis du numéro de ligne avec **yylineno** pour faciliter le débogage.
Nouveaux Types	Passage au type double pour supporter les nombres décimaux (ex: 3.14).
Tableaux	Gestion des structures de données indexées (ex: **tab[5] = 10;**).
Fonctions	Support de la définition et de l'appel de fonctions avec paramètres.

3. Opérateurs de comparaison

Objectif: Permettre à l'interpréteur de comprendre les symboles tels que **==**, **!=**, **<**, **>**.

3.1. Analyseur Lexical (lex.l)

lex.l

File Edit View

```
%{
#include "interpreteur.tab.h"
#include <stdlib.h>
#include <string.h>
%}

%option noyywrap

%%

[ \t\n\r]  {  }
"//".*     {  }

";"        { return FIN_INSTRUCTION; }
"="        { return AFFECTATION; }
"print"    { return AFFICHER; }
"("        { return OUVRE_PARENTHESE; }
")"        { return FERME_PARENTHESE; }
"+"        { return PLUS; }
"-"        { return MOINS; }
"*"        { return FOIS; }
"/"        { return DIVISE; }
"=="       { return EGAL; }
"!="       { return DIFF; }
"<"        { return INF; }
">"        { return SUP; }
"<="       { return INFEG; }
">="       { return SUPEG; }

[0-9]+     { yylval.reel = atof(yytext); return NOMBRE; }
[a-zA-Z_][a-zA-Z0-9_]* { yylval.chaine = strdup(yytext); return VARIABLE; }

.          { fprintf(stderr, "Caractere non reconnu : %s\n", yytext); }

%%
```

3.2. Analyseur Syntaxique (interpreteur.y)

interpreteur.y

File Edit View

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Table des symboles
typedef struct {
    char nom[32];
    double valeur;
} Symbole;

Symbole table[100];
int nb_symbols = 0;

double chercher_variable(char *nom) {
    for (int i = 0; i < nb_symbols; i++) {
        if (strcmp(table[i].nom, nom) == 0)
            return table[i].valeur;
    }
    return 0;
}

void definir_variable(char *nom, double valeur) {
    for (int i = 0; i < nb_symbols; i++) {
        if (strcmp(table[i].nom, nom) == 0) {
            table[i].valeur = valeur;
            return;
        }
    }
    strcpy(table[nb_symbols].nom, nom);
    table[nb_symbols].valeur = valeur;
    nb_symbols++;
}

int yylex(void);
void yyerror(const char *s) {
    fprintf(stderr, "Erreur : %s\n", s);
}
%}
```

```
%union {
    double reel;
    char *chaine;
}

%token <reel> NOMBRE
%token <chaine> VARIABLE
%token AFFECTATION FIN_INSTRUCTION
%token AFFICHER
%token OUVRE_PARENTHESE FERME_PARENTHESE
%token PLUS MOINS FOIS DIVISE
%token EGAL DIFF INF SUP INFEG SUPEG

%type <reel> expression

%left EGAL DIFF INF SUP INFEG SUPEG
%left PLUS MOINS
%left FOIS DIVISE

%%

programme:
    instruction_list
    ;

instruction_list:
    instruction_list instruction
    | /* vide */
    ;

instruction:
    VARIABLE AFFECTATION expression FIN_INSTRUCTION { definir_variable($1, $3); free($1); }
    | AFFICHER OUVRE_PARENTHESE expression FERME_PARENTHESE FIN_INSTRUCTION { printf("%g\n", $3); }
    ;

expression:
    NOMBRE { $$ = $1; }
    | VARIABLE { $$ = chercher_variable($1); free($1); }
    | expression PLUS expression { $$ = $1 + $3; }
    | expression MOINS expression { $$ = $1 - $3; }
    | expression FOIS expression { $$ = $1 * $3; }
    | expression DIVISE expression { if($3 != 0) $$ = $1 / $3; else { yyerror("Div par zero"); $$ = 0; } }
    | OUVRE_PARENTHESE expression FERME_PARENTHESE { $$ = $2; }
    | expression EGAL expression { $$ = ($1 == $3); }
    | expression DIFF expression { $$ = ($1 != $3); }
    | expression INF  expression { $$ = ($1 < $3); }
    | expression SUP  expression { $$ = ($1 > $3); }
    | expression INFEG expression { $$ = ($1 <= $3); }
    | expression SUPEG expression { $$ = ($1 >= $3); }
    ;

%%
```

3.3. Tests et exécution



```
Windows PowerShell
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> .\win_bison.exe -d interpreteur.y
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> .\win_flex lex.l
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> gcc main.c interpreteur.tab.c lex.yy.c -o compiler -lm
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> ./compiler test_comparaison.txt
=== Interpreteur du mini-langage (Etendu) ===
1
0
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet>
```

4. Opérateurs logiques

Objectif: Combiner des conditions à l'aide de **&& (ET)**, **|| (OU)**, **! (NON)**.

4.1. Analyseur Lexical (lex.l)

lex.l

File Edit View

```
%{
#include "interpreteur.tab.h"
#include <stdlib.h>
#include <string.h>
%}

%option noyywrap

%%

[ \t\n\r]  {  }
"//".*     {  }

";"        { return FIN_INSTRUCTION; }
"="        { return AFFECTATION; }
"print"    { return AFFICHER; }
"("        { return OUVRE_PARENTHESE; }
")"        { return FERME_PARENTHESE; }
"+"        { return PLUS; }
"-"        { return MOINS; }
"*"        { return FOIS; }
"/"        { return DIVISE; }
"=="       { return EGAL; }
"!="       { return DIFF; }
"<"        { return INF; }
">"        { return SUP; }
"<="       { return INFEG; }
">="       { return SUPEG; }
"&&"       { return ET; }
"||"       { return OU; }
"!"        { return NON; }

[0-9]+     { yylval.reel = atof(yytext); return NOMBRE; }
[a-zA-Z_][a-zA-Z0-9_]* { yylval.chaine = strdup(yytext); return VARIABLE; }

.          { fprintf(stderr, "Caractere non reconnu : %s\n", yytext); }

%%
```

4.2. Analyseur Syntaxique (interpreteur.y)

interpreteur.y

File Edit View

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Table des symboles
typedef struct {
    char nom[32];
    double valeur;
} Symbole;

Symbole table[100];
int nb_symbols = 0;

double chercher_variable(char *nom) {
    for (int i = 0; i < nb_symbols; i++) {
        if (strcmp(table[i].nom, nom) == 0)
            return table[i].valeur;
    }
    return 0;
}

void definir_variable(char *nom, double valeur) {
    for (int i = 0; i < nb_symbols; i++) {
        if (strcmp(table[i].nom, nom) == 0) {
            table[i].valeur = valeur;
            return;
        }
    }
    strcpy(table[nb_symbols].nom, nom);
    table[nb_symbols].valeur = valeur;
    nb_symbols++;
}

int yylex(void);
void yyerror(const char *s) {
    fprintf(stderr, "Erreur : %s\n", s);
}
%}
```

```
%union {
    double reel;
    char *chaine;
}

%token <reel> NOMBRE
%token <chaine> VARIABLE
%token AFFECTATION FIN_INSTRUCTION
%token AFFICHER
%token OUVRE_PARENTHESE FERME_PARENTHESE
%token PLUS MOINS FOIS DIVISE
%token EGAL DIFF INF SUP INFEG SUPEG
%token ET OU NON

%type <reel> expression

%left OU
%left ET
%left EGAL DIFF INF SUP INFEG SUPEG
%left PLUS MOINS
%left FOIS DIVISE
%right NON

%%

programme:
    instruction_list
    ;

instruction_list:
    instruction_list instruction
    | /* vide */
    ;

instruction:
    VARIABLE AFFECTATION expression FIN_INSTRUCTION { definir_variable($1, $3); free($1); }
    | AFFICHER OUVRE_PARENTHESE expression FERME_PARENTHESE FIN_INSTRUCTION { printf("%g\n", $3); }
    ;

expression:
    NOMBRE { $$ = $1; }
    | VARIABLE { $$ = chercher_variable($1); free($1); }
    | expression PLUS expression { $$ = $1 + $3; }
    | expression MOINS expression { $$ = $1 - $3; }
    | expression FOIS expression { $$ = $1 * $3; }
    | expression DIVISE expression { if($3 != 0) $$ = $1 / $3; else { yyerror("Div par zero"); $$ = 0; } }
    | OUVRE_PARENTHESE expression FERME_PARENTHESE { $$ = $2; }
    | expression EGAL expression { $$ = ($1 == $3); }
    | expression DIFF expression { $$ = ($1 != $3); }
    | expression INF  expression { $$ = ($1 < $3); }
    | expression SUP  expression { $$ = ($1 > $3); }
    | expression INFEG expression { $$ = ($1 <= $3); }
    | expression SUPEG expression { $$ = ($1 >= $3); }
    | expression ET expression { $$ = ($1 && $3); }
    | expression OU expression { $$ = ($1 || $3); }
    | NON expression           { $$ = !($2); }
    ;

%%
```

4.3. Tests et exécution

test_logique.txt

File Edit View

```
a = 10;
b = 5;
c = 20;

res1 = (a > b) && (c > a);
print(res1); // doit afficher 1 ( vrai )

res2 = (a < b) || (c > 100);
print(res2); // doit afficher 0 ( faux )

res3 = !(a == b);
print(res3); // doit afficher 1 ( vrai )
```

```
Windows PowerShell
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> .\win_bison.exe -d interpreteur.y
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> .\win_flex lex.l
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> gcc main.c interpreteur.tab.c lex.yy.c -o compiler -lm
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> ./compiler test_logique.txt
=== Interpreteur du mini-langage (Etendu) ===
1
0
1
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet>
```

5. Structures de contrôle

Objectif: Implémenter les structures décisionnelles **(si...alors...sinon)** et les structures itératives **(tant que et repeter...jusqua)**.

5.1. Analyseur Lexical (lex.l)

lex.l

File Edit View

```
%{
#include "interpreteur.tab.h"
#include <stdlib.h>
#include <string.h>
%}

%option noyywrap

%%

[ \t\n\r]  {  }
"//".*     {  }

";"        { return FIN_INSTRUCTION; }
"="        { return AFFECTATION; }
"si"       { return SI; }
"alors"    { return ALORS; }
"sinon"    { return SINON; }
"fin"      { return FIN; }
"tant que" { return TANTQUE; }
"repeter"  { return REPETER; }
"jusqua"   { return JUSQUA; }
"print"    { return AFFICHER; }
"("        { return OUVRE_PARENTHESE; }
")"        { return FERME_PARENTHESE; }
"+"        { return PLUS; }
"-"        { return MOINS; }
"*"        { return FOIS; }
"/"        { return DIVISE; }
"=="       { return EGAL; }
"!="       { return DIFF; }
"<"        { return INF; }
">"        { return SUP; }
"<="       { return INFEG; }
">="       { return SUPEG; }
"&&"       { return ET; }
"||"       { return OU; }
"!"        { return NON; }

[0-9]+     { yylval.reel = atof(yytext); return NOMBRE; }
[a-zA-Z_][a-zA-Z0-9_]* { yylval.chaine = strdup(yytext); return VARIABLE; }

.          { fprintf(stderr, "Caractere non reconnu : %s\n", yytext); }

%%
```

5.2. Analyseur Syntaxique (interpreteur.y)

interpreteur.y

File Edit View

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int execute_flag = 1;

// Table des symboles
typedef struct {
    char nom[32];
    double valeur;
} Symbole;

Symbole table[100];
int nb_symbols = 0;

double chercher_variable(char *nom) {
    for (int i = 0; i < nb_symbols; i++) {
        if (strcmp(table[i].nom, nom) == 0)
            return table[i].valeur;
    }
    return 0;
}

void definir_variable(char *nom, double valeur) {
    for (int i = 0; i < nb_symbols; i++) {
        if (strcmp(table[i].nom, nom) == 0) {
            table[i].valeur = valeur;
            return;
        }
    }
    strcpy(table[nb_symbols].nom, nom);
    table[nb_symbols].valeur = valeur;
    nb_symbols++;
}

int yylex(void);
void yyerror(const char *s) {
fprintf(stderr, "Erreur : %s\n", s);
}
%}
```

```
%union {
    double reel;
    char *chaine;
}

%token <reel> NOMBRE
%token <chaine> VARIABLE
%token AFFECTATION FIN_INSTRUCTION
%token AFFICHER
%token OUVRE_PARENTHESE FERME_PARENTHESE
%token PLUS MOINS FOIS DIVISE
%token EGAL DIFF INF SUP INFEG SUPEG
%token ET OU NON
%token SI ALORS SINON FIN TANTQUE REPETER JUSQUA

%type <reel> expression

%left OU
%left ET
%left EGAL DIFF INF SUP INFEG SUPEG
%left PLUS MOINS
%left FOIS DIVISE
%right NON

%%

programme:
    instruction_list
    ;

instruction_list:
    instruction_list instruction
    | /* vide */
    ;

instruction:
    VARIABLE AFFECTATION expression FIN_INSTRUCTION { if(execute_flag) definir_variable($1, $3); free($1); }
    | AFFICHER OUVRE_PARENTHESE expression FERME_PARENTHESE FIN_INSTRUCTION { if(execute_flag) printf("%g\n", $3); }
    | conditionnelle
    | boucle
    ;

conditionnelle:
    SI OUVRE_PARENTHESE expression FERME_PARENTHESE ALORS { if($3 == 0) execute_flag = 0; }
    instruction_list suite_conditionnelle FIN { execute_flag = 1; printf("[Info] Bloc SI-SINON reconnu\n"); }
    ;

suite_conditionnelle:
    /* vide */
    | SINON { if(execute_flag == 1) execute_flag = 0; else execute_flag = 1; }
    instruction_list
    ;

boucle:
    TANTQUE OUVRE_PARENTHESE expression FERME_PARENTHESE ALORS instruction_list FIN {
        printf("[Info] Boucle TANT QUE reconnue\n");
    }
    | REPETER instruction_list JUSQUA OUVRE_PARENTHESE expression FERME_PARENTHESE FIN_INSTRUCTION {
        printf("[Info] Boucle REPETER reconnue\n");
    }
    ;

expression:
    NOMBRE { $$ = $1; }
    | VARIABLE { $$ = chercher_variable($1); free($1); }
    | expression PLUS expression { $$ = $1 + $3; }
    | expression MOINS expression { $$ = $1 - $3; }
    | expression FOIS expression { $$ = $1 * $3; }
    | expression DIVISE expression { if($3 != 0) $$ = $1 / $3; else { yyerror("Div par zero"); $$ = 0; } }
    | OUVRE_PARENTHESE expression FERME_PARENTHESE { $$ = $2; }
    | expression EGAL expression { $$ = ($1 == $3); }
    | expression DIFF expression { $$ = ($1 != $3); }
    | expression INF  expression { $$ = ($1 < $3); }
    | expression SUP  expression { $$ = ($1 > $3); }
    | expression INFEG expression { $$ = ($1 <= $3); }
    | expression SUPEG expression { $$ = ($1 >= $3); }
    | expression ET expression { $$ = ($1 && $3); }
    | expression OU expression { $$ = ($1 || $3); }
    | NON expression           { $$ = !($2); }
    ;

%%
```

5.3. Tests et exécution

Notre interpréteur actuel est un **interpréteur syntaxique**, ce qui signifie qu'il exécute les commandes dès qu'il les lit. Lorsque l'analyseur syntaxique rencontre une instruction `**si-sinon**`, il exécute toutes les instructions `**print**` rencontrées, quel que soit le résultat de la condition.

test_structure.txt

File Edit View

```
i = 4;

si (i > 3) alors
    print(1);
sinon
    print(0);
fin
// doit afficher 1

tant que (i < 5) alors
    i = i + 1;
    print(i);
fin
// doit afficher 5

repeter
    i = i - 1;
    print(i);
jusqua (i == 0); // doit afficher 4
```

Windows PowerShell

```
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> .\win_bison.exe -d interpreteur.y
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> .\win_flex lex.l
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> gcc main.c interpreteur.tab.c lex.yy.c -o compiler -lm
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> ./compiler test_structure.txt
=== Interpreteur du mini-langage (Etendu) ===
1
[Info] Bloc SI-SINON reconnu
5
[Info] Boucle TANT QUE reconnue
4
[Info] Boucle REPETER reconnue
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet>
```

C'est pourquoi nous avons obtenu le résultat **4**.

6. Gestion d'erreurs

Objectif: Améliorer les messages d'erreur en y incluant le numéro de ligne avec **yylineno**.

6.1. Analyseur Lexical (lex.l)

lex.l

File Edit View

```
%{
#include "interpreteur.tab.h"
#include <stdlib.h>
#include <string.h>
%}

%option noyywrap
%option yylineno

%%

[ \t\n\r]  {  }
"//".*     {  }

";"        { return FIN_INSTRUCTION; }
"="        { return AFFECTATION; }
"si"       { return SI; }
"alors"    { return ALORS; }
"sinon"    { return SINON; }
"fin"      { return FIN; }
"tant que" { return TANTQUE; }
"repeter"  { return REPETER; }
"jusqua"   { return JUSQUA; }
"print"    { return AFFICHER; }
"("        { return OUVRE_PARENTHESE; }
")"        { return FERME_PARENTHESE; }
"+"        { return PLUS; }
"-"        { return MOINS; }
"*"        { return FOIS; }
"/"        { return DIVISE; }
"=="       { return EGAL; }
"!="       { return DIFF; }
"<"        { return INF; }
">"        { return SUP; }
"<="       { return INFEG; }
">="       { return SUPEG; }
"&&"       { return ET; }
"||"       { return OU; }
"!"        { return NON; }

[0-9]+     { yylval.reel = atof(yytext); return NOMBRE; }
[a-zA-Z_][a-zA-Z0-9_]* { yylval.chaine = strdup(yytext); return VARIABLE; }

.          { fprintf(stderr, "Erreur Lexicale a la ligne %d : Caractere inconnu [%s]\n", yylineno, yytext); }

%%
```

6.2. Analyseur Syntaxique (interpreteur.y)

interpreteur.y

File Edit View

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int execute_flag = 1;

// Table des symboles
typedef struct {
    char nom[32];
    double valeur;
} Symbole;

Symbole table[100];
int nb_symbols = 0;

double chercher_variable(char *nom) {
    for (int i = 0; i < nb_symbols; i++) {
        if (strcmp(table[i].nom, nom) == 0)
            return table[i].valeur;
    }
    return 0;
}

void definir_variable(char *nom, double valeur) {
    for (int i = 0; i < nb_symbols; i++) {
        if (strcmp(table[i].nom, nom) == 0) {
            table[i].valeur = valeur;
            return;
        }
    }
    strcpy(table[nb_symbols].nom, nom);
    table[nb_symbols].valeur = valeur;
    nb_symbols++;
}

int yylex(void);
extern int yylineno;
void yyerror(const char *s) {
    fprintf(stderr, "Erreur ligne %d : %s\n", yylineno, s);
}
%}
```

```
%union {
    double reel;
    char *chaine;
}

%token <reel> NOMBRE
%token <chaine> VARIABLE
%token AFFECTATION FIN_INSTRUCTION
%token AFFICHER
%token OUVRE_PARENTHESE FERME_PARENTHESE
%token PLUS MOINS FOIS DIVISE
%token EGAL DIFF INF SUP INFEG SUPEG
%token ET OU NON
%token SI ALORS SINON FIN TANTQUE REPETER JUSQUA

%type <reel> expression

%left OU
%left ET
%left EGAL DIFF INF SUP INFEG SUPEG
%left PLUS MOINS
%left FOIS DIVISE
%right NON

%%

programme:
    instruction_list
    ;

instruction_list:
    instruction_list instruction
    | /* vide */
    ;

instruction:
    VARIABLE AFFECTATION expression FIN_INSTRUCTION { if(execute_flag) definir_variable($1, $3); free($1); }
    | AFFICHER OUVRE_PARENTHESE expression FERME_PARENTHESE FIN_INSTRUCTION { if(execute_flag) printf("%g\n", $3); }
    | conditionnelle
    | boucle
    | error FIN_INSTRUCTION { yyerrok; }
    ;

conditionnelle:
    SI OUVRE_PARENTHESE expression FERME_PARENTHESE ALORS { if($3 == 0) execute_flag = 0; }
    instruction_list suite_conditionnelle FIN { execute_flag = 1; printf("[Info] Bloc SI-SINON reconnu\n"); }
    ;

suite_conditionnelle:
    /* vide */
    | SINON { if(execute_flag == 1) execute_flag = 0; else execute_flag = 1; }
    instruction_list
    ;

boucle:
    TANTQUE OUVRE_PARENTHESE expression FERME_PARENTHESE ALORS instruction_list FIN {
        printf("[Info] Boucle TANT QUE reconnue\n");
    }
    | REPETER instruction_list JUSQUA OUVRE_PARENTHESE expression FERME_PARENTHESE FIN_INSTRUCTION {
        printf("[Info] Boucle REPETER reconnue\n");
    }
    ;

expression:
    NOMBRE { $$ = $1; }
    | VARIABLE { $$ = chercher_variable($1); free($1); }
    | expression PLUS expression { $$ = $1 + $3; }
    | expression MOINS expression { $$ = $1 - $3; }
    | expression FOIS expression { $$ = $1 * $3; }
    | expression DIVISE expression { if($3 != 0) $$ = $1 / $3; else { yyerror("division par zero"); $$ = 0; } }
    | OUVRE_PARENTHESE expression FERME_PARENTHESE { $$ = $2; }
    | expression EGAL expression { $$ = ($1 == $3); }
    | expression DIFF expression { $$ = ($1 != $3); }
    | expression INF  expression { $$ = ($1 < $3); }
    | expression SUP  expression { $$ = ($1 > $3); }
    | expression INFEG expression { $$ = ($1 <= $3); }
    | expression SUPEG expression { $$ = ($1 >= $3); }
    | expression ET expression { $$ = ($1 && $3); }
    | expression OU expression { $$ = ($1 || $3); }
    | NON expression           { $$ = !($2); }
    ;

%%
```

6.3. Tests et exécution

test_erreur.txt

File Edit View

```
a = 10;
b = 0;

// Test de division par zéro
print(a / 0);

// Test de division par une variable égale à zéro
res = a / b;

// Vérification que le programme continue après l'erreur
print(111);
```

Windows PowerShell

```
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> .\win_bison.exe -d interpreteur.y
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> .\win_flex lex.l
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> gcc main.c interpreteur.tab.c lex.yy.c -o compiler -lm
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> ./compiler test_erreur.txt
=== Interpreteur du mini-langage (Etendu) ===
Erreur ligne 5 : division par zero
0
Erreur ligne 8 : division par zero
111
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet>
```

7. Nouveaux types (réels)

Objectif: Passer de **int** à **double** pour prendre en charge des nombres comme **3,14**

7.1. Analyseur Lexical (lex.l)

lex.l

File Edit View

```
%{
#include "interpreteur.tab.h"
#include <stdlib.h>
#include <string.h>
%}

%option noyywrap
%option yylineno

%%

[ \t\n\r]  {  }
"//".*     {  }

";"        { return FIN_INSTRUCTION; }
"="        { return AFFECTATION; }
"si"       { return SI; }
"alors"    { return ALORS; }
"sinon"    { return SINON; }
"fin"      { return FIN; }
"tant que" { return TANTQUE; }
"repeter"  { return REPETER; }
"jusqua"   { return JUSQUA; }
"print"    { return AFFICHER; }
"("        { return OUVRE_PARENTHESE; }
")"        { return FERME_PARENTHESE; }
"+"        { return PLUS; }
"-"        { return MOINS; }
"*"        { return FOIS; }
"/"        { return DIVISE; }
"=="       { return EGAL; }
"!="       { return DIFF; }
"<"        { return INF; }
">"        { return SUP; }
"<="       { return INFEG; }
">="       { return SUPEG; }
"&&"       { return ET; }
"||"       { return OU; }
"!"        { return NON; }

[0-9]+(\.[0-9]+)?      { yylval.reel = atof(yytext); return NOMBRE; }
[a-zA-Z_][a-zA-Z0-9_]* { yylval.chaine = strdup(yytext); return VARIABLE; }

.          { fprintf(stderr, "Erreur Lexicale a la ligne %d : Caractere inconnu [%s]\n", yylineno, yytext); }

%%
```

7.2. Analyseur Syntaxique (interpreteur.y)

il restera tel quel

7.3. Tests et exécution

test_reels.txt

File Edit View

```
pi = 3.14;
rayon = 10.5;
surface = pi * (rayon * rayon);

si (surface > 100.0) alors
    print(surface);   // doit afficher 346.185 ( 3.14 * (10.5 * 10.5) )
sinon
    print(0.0);
fin
```

```
Windows PowerShell
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> .\win_bison.exe -d interpreteur.y
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> .\win_flex lex.l
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> gcc main.c interpreteur.tab.c lex.yy.c -o compiler -lm
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> ./compiler test_reels.txt
=== Interpreteur du mini-langage (Etendu) ===
346.185
[Info] Bloc SI-SINON reconnu
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet>
```

8. Tableaux

Objectif: Ajouter la prise en charge des tableaux simples **tab[i]**.

8.1. Analyseur Lexical (lex.l)

lex.l

File Edit View

```
%{
#include "interpreteur.tab.h"
#include <stdlib.h>
#include <string.h>
%}

%option noyywrap
%option yylineno

%%

[ \t\n\r]  {  }
"//".*     {  }

";"        { return FIN_INSTRUCTION; }
"="        { return AFFECTATION; }
"si"       { return SI; }
"alors"    { return ALORS; }
"sinon"    { return SINON; }
"fin"      { return FIN; }
"tant que" { return TANTQUE; }
"repeter"  { return REPETER; }
"jusqua"   { return JUSQUA; }
"print"    { return AFFICHER; }
"("        { return OUVRE_PARENTHESE; }
")"        { return FERME_PARENTHESE; }
"["        { return CROCHET_OUV; }
"]"        { return CROCHET_FER; }
"+"        { return PLUS; }
"-"        { return MOINS; }
"*"        { return FOIS; }
"/"        { return DIVISE; }
"=="       { return EGAL; }
"!="       { return DIFF; }
"<"        { return INF; }
">"        { return SUP; }
"<="       { return INFEG; }
">="       { return SUPEG; }
"&&"       { return ET; }
"||"       { return OU; }
"!"        { return NON; }

[0-9]+(\.[0-9]+)?      { yylval.reel = atof(yytext); return NOMBRE; }
[a-zA-Z_][a-zA-Z0-9_]* { yylval.chaine = strdup(yytext); return VARIABLE; }

.          { fprintf(stderr, "Erreur Lexicale a la ligne %d : Caractere inconnu [%s]\n", yylineno, yytext); }

%%
```

8.2. Analyseur Syntaxique (interpreteur.y)

interpreteur.y

File Edit View

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int execute_flag = 1;

// Table des symboles
typedef struct {
    char nom[32];
    double valeur;
    double valeurs[100];
    int est_tableau;
} Symbole;

Symbole table[100];
int nb_symbols = 0;

double chercher_variable(char *nom, int index) {
    for (int i = 0; i < nb_symbols; i++) {
        if (strcmp(table[i].nom, nom) == 0) {
            if (table[i].est_tableau) return table[i].valeurs[index];
            return table[i].valeur;
        }
    }
    return 0;
}

void definir_variable(char *nom, double valeur) {
    for (int i = 0; i < nb_symbols; i++) {
        if (strcmp(table[i].nom, nom) == 0) {
            table[i].valeur = valeur;
            table[i].est_tableau = 0;
            return;
        }
    }
    strcpy(table[nb_symbols].nom, nom);
    table[nb_symbols].valeur = valeur;
    table[nb_symbols].est_tableau = 0;
    nb_symbols++;
}
```

```
void definir_element_tableau(char *nom, int index, double valeur) {
    for (int i = 0; i < nb_symbols; i++) {
        if (strcmp(table[i].nom, nom) == 0) {
            table[i].valeurs[index] = valeur;
            table[i].est_tableau = 1;
            return;
        }
    }
    strcpy(table[nb_symbols].nom, nom);
    table[nb_symbols].valeurs[index] = valeur;
    table[nb_symbols].est_tableau = 1;
    nb_symbols++;
}
```

```
int yylex(void);
extern int yylineno;
void yyerror(const char *s) {
    fprintf(stderr, "Erreur ligne %d : %s\n", yylineno, s);
}
%}

%union {
    double reel;
    char *chaine;
}

%token <reel> NOMBRE
%token <chaine> VARIABLE
%token AFFECTATION FIN_INSTRUCTION
%token AFFICHER
%token OUVRE_PARENTHESE FERME_PARENTHESE
%token CROCHET_OUV CROCHET_FER
%token PLUS MOINS FOIS DIVISE
%token EGAL DIFF INF SUP INFEG SUPEG
%token ET OU NON
%token SI ALORS SINON FIN TANTQUE REPETER JUSQUA

%type <reel> expression

%left OU
%left ET
%left EGAL DIFF INF SUP INFEG SUPEG
%left PLUS MOINS
%left FOIS DIVISE
%right NON

%%

programme:
    instruction_list
    ;

instruction_list:
    instruction_list instruction
    | /* vide */
    ;
```

```
instruction:
    VARIABLE AFFECTATION expression FIN_INSTRUCTION {
        if(execute_flag) definir_variable($1, $3);
        free($1);
    }
    | VARIABLE CROCHET_OUV expression CROCHET_FER AFFECTATION expression FIN_INSTRUCTION {
        if(execute_flag) definir_element_tableau($1, (int)$3, $6);
        free($1);
    }
    | AFFICHER OUVRE_PARENTHESE expression FERME_PARENTHESE FIN_INSTRUCTION { if(execute_flag) printf("%g\n", $3); }
    | conditionnelle
    | boucle
    | error FIN_INSTRUCTION { yyerrok; }
    ;

conditionnelle:
    SI OUVRE_PARENTHESE expression FERME_PARENTHESE ALORS { if($3 == 0) execute_flag = 0; }
    instruction_list suite_conditionnelle FIN { execute_flag = 1; printf("[Info] Bloc SI-SINON reconnu\n"); }
    ;

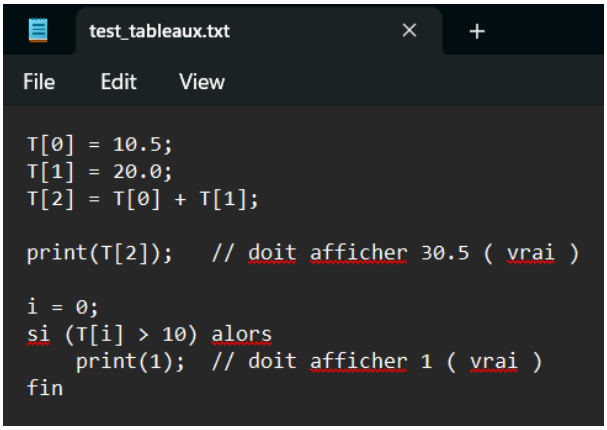
suite_conditionnelle:
    /* vide */
    | SINON { if(execute_flag == 1) execute_flag = 0; else execute_flag = 1; }
    instruction_list
    ;

boucle:
    TANTQUE OUVRE_PARENTHESE expression FERME_PARENTHESE ALORS instruction_list FIN {
        printf("[Info] Boucle TANT QUE reconnue\n");
    }
    | REPETER instruction_list JUSQUA OUVRE_PARENTHESE expression FERME_PARENTHESE FIN_INSTRUCTION {
        printf("[Info] Boucle REPETER reconnue\n");
    }
    ;

expression:
    NOMBRE { $$ = $1; }
    | VARIABLE { $$ = chercher_variable($1, 0); free($1); }
    | VARIABLE CROCHET_OUV expression CROCHET_FER { $$ = chercher_variable($1, (int)$3); free($1); }
    | expression PLUS expression { $$ = $1 + $3; }
    | expression MOINS expression { $$ = $1 - $3; }
    | expression FOIS expression { $$ = $1 * $3; }
    | expression DIVISE expression { if($3 != 0) $$ = $1 / $3; else { yyerror("division par zero"); $$ = 0; } }
    | OUVRE_PARENTHESE expression FERME_PARENTHESE { $$ = $2; }
    | expression EGAL expression { $$ = ($1 == $3); }
    | expression DIFF expression { $$ = ($1 != $3); }
    | expression INF  expression { $$ = ($1 < $3); }
    | expression SUP  expression { $$ = ($1 > $3); }
    | expression INFEG expression { $$ = ($1 <= $3); }
    | expression SUPEG expression { $$ = ($1 >= $3); }
    | expression ET expression { $$ = ($1 && $3); }
    | expression OU expression { $$ = ($1 || $3); }
    | NON expression           { $$ = !($2); }
    ;

%%
```

8.3. Tests et exécution



```
Windows PowerShell
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> .\win_bison.exe -d interpreteur.y
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> .\win_flex lex.l
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> gcc main.c interpreteur.tab.c lex.yy.c -o compiler -lm
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> ./compiler test_tableaux.txt
=== Interpreteur du mini-langage (Etendu) ===
30.5
1
[Info] Bloc SI-SINON reconnu
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet>
```

9. Fonctions

Objectif: Prise en charge des fonctions mathématiques intégrées (telles que **sqrt**) ou définition de fonctions simples.

9.1. Analyseur Lexical (lex.l)

lex.l

File Edit View

```
%{
#include "interpreteur.tab.h"
#include <stdlib.h>
#include <string.h>
%}

%option noyywrap
%option yylineno

%%

[ \t\n\r]   {  }
"//".*      {  }

","         { return VIRGULE; }
";"         { return FIN_INSTRUCTION; }
"="         { return AFFECTATION; }
"si"        { return SI; }
"alors"     { return ALORS; }
"sinon"     { return SINON; }
"fin"       { return FIN; }
"tant que"  { return TANTQUE; }
"repeter"   { return REPETER; }
"jusqua"    { return JUSQUA; }
"print"     { return AFFICHER; }
"fonction"  { return FONCTION; }
"retourner" { return RETOURNER; }
"("         { return OUVRE_PARENTHESE; }
")"         { return FERME_PARENTHESE; }
"["         { return CROCHET_OUV; }
"]"         { return CROCHET_FER; }
"+"         { return PLUS; }
"-"         { return MOINS; }
"*"         { return FOIS; }
"/"         { return DIVISE; }
"=="        { return EGAL; }
"!="        { return DIFF; }
"<"         { return INF; }
">"         { return SUP; }
"<="        { return INFEG; }
">="        { return SUPEG; }
"&&"        { return ET; }
"||"        { return OU; }
"!"         { return NON; }

[0-9]+(\.[0-9]+)?      { yylval.reel = atof(yytext); return NOMBRE; }
[a-zA-Z_][a-zA-Z0-9_]* { yylval.chaine = strdup(yytext); return VARIABLE; }

.           { fprintf(stderr, "Erreur Lexicale a la ligne %d : Caractere inconnu [%s]\n", yylineno, yytext); }

%%
```

9.2. Analyseur Syntaxique (interpreteur.y)

interpreteur.y

File Edit View

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
int execute_flag = 1;

// Table des symboles
typedef struct {
    char nom[32];
    double valeur;
    double valeurs[100];
    int est_tableau;
} Symbole;

Symbole table[100];
int nb_symbols = 0;

double executer_fonction(char *nom, double param) {
    if (strcmp(nom, "carre") == 0) return param * param;
    if (strcmp(nom, "racine") == 0) return sqrt(param);
    if (strcmp(nom, "abs") == 0) return fabs(param);
    printf("Erreur : Fonction '%s' non definie\n", nom);
    return 0;
}

double chercher_variable(char *nom, int index) {
    for (int i = 0; i < nb_symbols; i++) {
        if (strcmp(table[i].nom, nom) == 0) {
            if (table[i].est_tableau) return table[i].valeurs[index];
            return table[i].valeur;
        }
    }
    return 0;
}

void definir_variable(char *nom, double valeur) {
    for (int i = 0; i < nb_symbols; i++) {
        if (strcmp(table[i].nom, nom) == 0) {
            table[i].valeur = valeur;
            table[i].est_tableau = 0;
            return;
        }
    }
    strcpy(table[nb_symbols].nom, nom);
    table[nb_symbols].valeur = valeur;
    table[nb_symbols].est_tableau = 0;
    nb_symbols++;
}
```

```
void definir_element_tableau(char *nom, int index, double valeur) {
    for (int i = 0; i < nb_symbols; i++) {
        if (strcmp(table[i].nom, nom) == 0) {
            table[i].valeurs[index] = valeur;
            table[i].est_tableau = 1;
            return;
        }
    }
    strcpy(table[nb_symbols].nom, nom);
    table[nb_symbols].valeurs[index] = valeur;
    table[nb_symbols].est_tableau = 1;
    nb_symbols++;
}

int yylex(void);
extern int yylineno;
void yyerror(const char *s) {
    fprintf(stderr, "Erreur ligne %d : %s\n", yylineno, s);
}
%}

%union {
    double reel;
    char *chaine;
}

%token <reel> NOMBRE
%token <chaine> VARIABLE
%token AFFECTATION FIN_INSTRUCTION
%token AFFICHER
%token OUVRE_PARENTHESE FERME_PARENTHESE
%token CROCHET_OUV CROCHET_FER
%token FONCTION RETOURNER VIRGULE
%token PLUS MOINS FOIS DIVISE
%token EGAL DIFF INF SUP INFEG SUPEG
%token ET OU NON
%token SI ALORS SINON FIN TANTQUE REPETER JUSQUA

%type <reel> expression

%left OU
%left ET
%left EGAL DIFF INF SUP INFEG SUPEG
%left PLUS MOINS
%left FOIS DIVISE
%right NON

%%

programme:
    instruction_list
    ;

instruction_list:
    instruction_list instruction
    | /* vide */
    ;
```

```
instruction:
    VARIABLE AFFECTATION expression FIN_INSTRUCTION {
        if(execute_flag) definir_variable($1, $3);
        free($1);
    }
    | VARIABLE CROCHET_OUV expression CROCHET_FER AFFECTATION expression FIN_INSTRUCTION {
        if(execute_flag) definir_element_tableau($1, (int)$3, $6);
        free($1);
    }
    | AFFICHER OUVRE_PARENTHESE expression FERME_PARENTHESE FIN_INSTRUCTION { if(execute_flag) printf("%g\n", $3); }
    | conditionnelle
    | boucle
    | error FIN_INSTRUCTION { yyerrok; }
    ;

conditionnelle:
    SI OUVRE_PARENTHESE expression FERME_PARENTHESE ALORS { if($3 == 0) execute_flag = 0; }
    instruction_list suite_conditionnelle FIN { execute_flag = 1; printf("[Info] Bloc SI-SINON reconnu\n"); }
    ;

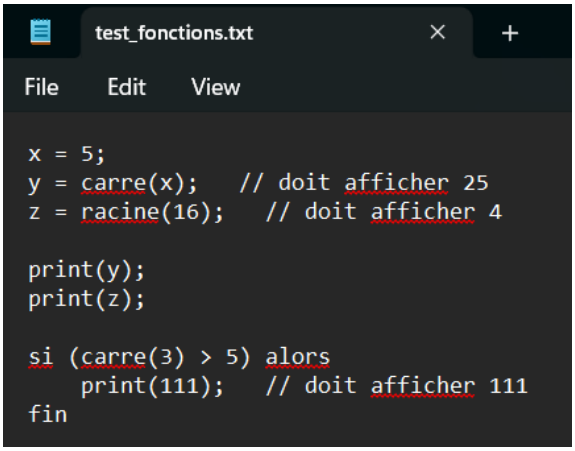
suite_conditionnelle:
    /* vide */
    | SINON { if(execute_flag == 1) execute_flag = 0; else execute_flag = 1; }
    instruction_list
    ;

boucle:
    TANTQUE OUVRE_PARENTHESE expression FERME_PARENTHESE ALORS instruction_list FIN {
        printf("[Info] Boucle TANT QUE reconnue\n");
    }
    | REPETER instruction_list JUSQUA OUVRE_PARENTHESE expression FERME_PARENTHESE FIN_INSTRUCTION {
        printf("[Info] Boucle REPETER reconnue\n");
    }
    ;

expression:
    NOMBRE { $$ = $1; }
    | VARIABLE { $$ = chercher_variable($1, 0); free($1); }
    | VARIABLE CROCHET_OUV expression CROCHET_FER { $$ = chercher_variable($1, (int)$3); free($1); }
    | VARIABLE OUVRE_PARENTHESE expression FERME_PARENTHESE { $$ = executer_fonction($1, $3); free($1); }
    | expression PLUS expression { $$ = $1 + $3; }
    | expression MOINS expression { $$ = $1 - $3; }
    | expression FOIS expression { $$ = $1 * $3; }
    | expression DIVISE expression { if($3 != 0) $$ = $1 / $3; else { yyerror("division par zero"); $$ = 0; } }
    | OUVRE_PARENTHESE expression FERME_PARENTHESE { $$ = $2; }
    | expression EGAL expression { $$ = ($1 == $3); }
    | expression DIFF expression { $$ = ($1 != $3); }
    | expression INF  expression { $$ = ($1 < $3); }
    | expression SUP  expression { $$ = ($1 > $3); }
    | expression INFEG expression { $$ = ($1 <= $3); }
    | expression SUPEG expression { $$ = ($1 >= $3); }
    | expression ET expression { $$ = ($1 && $3); }
    | expression OU expression { $$ = ($1 || $3); }
    | NON expression           { $$ = !($2); }
    ;

%%
```

9.3. Tests et exécution



```
Windows PowerShell
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> .\win_bison.exe -d interpreteur.y
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> .\win_flex lex.l
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> gcc main.c interpreteur.tab.c lex.yy.c -o compiler -lm
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet> ./compiler test_fonctions.txt
=== Interpreteur du mini-langage (Etendu) ===
25
4
111
[Info] Bloc SI-SINON reconnu
PS C:\Users\ahmed\Desktop\TP Compilation\mini_projet>
```

Limitation actuelle: L'interpréteur actuel supporte les fonctions **pré-définies** (built-in) comme **carre()** et **racine()** car leur logique est codée en **C**. Cependant, la définition de **nouvelles fonctions** par l'utilisateur directement dans le fichier texte nécessite une structure de données plus complexe (comme un **Arbre de Syntaxe Abstraite - AST**) pour stocker le corps de la fonction et le réexécuter lors de l'appel.

10. Outils de Débogage et Validation

Pour faciliter la détection des erreurs syntaxiques complexes et suivre le comportement de l'automate de **Bison** (états de shift/reduce), nous avons activé le mode debug:

main.c

```c
#include <stdio.h>
#include <stdlib.h>

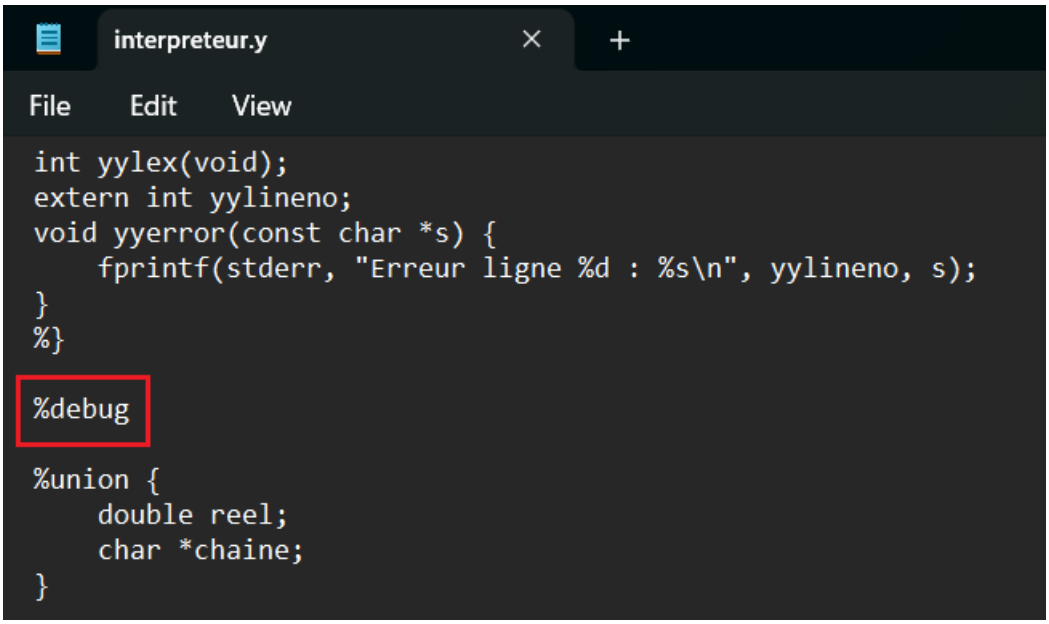
extern int yyparse(void);
extern FILE *yyin;
extern int yydebug;

int main(int argc, char **argv) {
    yydebug = 1;
    if (argc > 1) {
        yyin = fopen(argv[1], "r");
        if (!yyin) {
            fprintf(stderr, "Impossible d'ouvrir %s\n", argv[1]);
            return 1;
        }
    } else {
        yyin = stdin;
    }
    printf("=== Interpreteur du mini-langage (Etendu) ===\n");
    if (yyin == stdin) {
        printf("Entrez vos instructions (Ctrl+D pour quitter) :\n");
    }

    yyparse();

    if (yyin != stdin) fclose(yyin);
    return 0;
}
```

Il faut indiquer à **Bison** de générer le code de débogage dans le fichier de l'interpréteur en ajoutant la commande **%debug** dans la section des déclarations du fichier **interpreteur.y**:



Cette étape a été cruciale pour valider la hiérarchie des expressions et la priorité des opérateurs.

Le mode débogage de **Bison** affiche une **“feuille de route”** qui vous indique comment l'interpréteur a analysé les fichiers de test. Pour exécuter le programme sans afficher ces détails, remplacez **“yydebug=0;”** dans le fichier **“main.c”** et recompilez (avec **GCC**); vous obtiendrez alors uniquement le résultat final.

11. Conclusion

L'interpréteur développé est désormais capable de gérer des calculs complexes avec des **nombres réels** et **tableaux**, ainsi que des **conditions logiques imbriquées**. Grâce à l'intégration du suivi des lignes et du mode **%debug** de Bison, il offre une aide efficace au débogage.